Acceso a bases de datos y programación concurrente

UNICAM – CUR Matagalpa – Departamento de Ciencias, Tecnología y Salud

Fecha: octubre, 2026



UNIVERSIDAD NACIONAL AUTÓNOMA DE NICARAGUA, MANAGUA
UNAN-MANAGUA



Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa – Departamento de Ciencias, Tecnología y Salud

Manual didáctico Encuentro N°2: Acceso a bases de datos y programación concurrente

Autor: Heydy Judelki Mairena Palacios

Componente: Programación de aplicaciones de escritorio

Carrera: Ingeniería de Sistemas

Año: II año – II semestre

Octubre, 2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	**Revisado por:**	**Autorizado por:**
Heydy Judelki Mairena Palacios	Erick Lanzas	Erick Lanzas

Presentación del material

Bienvenido y bienvenida al segundo encuentro del componente Programación de Aplicaciones de Escritorio. En el Encuentro 1 construyó AgroStock v1, cuya lista de productos vivía en la memoria y se perdía al cerrar la aplicación. En este módulo —que aborda las Unidades II y III del programa— AgroStock deja de olvidar sus datos: los guardaremos en una base de datos SQLite y realizaremos las operaciones CRUD desde la interfaz.

A lo largo de estas páginas comprenderá cómo se organiza una base de datos relacional, cómo se consulta con SQL, cómo se conecta una aplicación C# con ADO.NET y cómo se ejecutan procesos largos (como importar cientos de productos) en segundo plano, con sincronización y cancelación, para que la ventana nunca diga «No responde». Al final tendrá AgroStock v2 funcionando, la segunda etapa del proyecto integrador (v1 → v2 → v3).

Horas del programa para este módulo: HT 10 · HP 20 · HTI 15 (total 45 h).

Objetivo de aprendizaje del encuentro

- Conectar una aplicación de escritorio a una base de datos SQLite y realizar operaciones CRUD mediante SQL parametrizado y clases de acceso a datos.
- Implementar operaciones en segundo plano con hilos y tareas, sincronización y detención controlada, manteniendo la interfaz responsiva.

Contenidos de este encuentro

- **Unidad II. Acceso a bases de datos:** introducción a la programación con bases de datos; el lenguaje SQL; clases para la gestión de bases de datos; controles de acceso a bases de datos; operaciones CRUD sobre una base de datos.
- **Unidad III. Programación concurrente:** introducción a la programación concurrente; clases para trabajar con hilos; estados de un hilo; mecanismos de sincronización; detención controlada de hilos.

A. Entrada temática

Vivencia

El viernes, en el agroservicio de doña Martha, se fue la luz a media mañana. Cuando volvió, AgroStock v1 abrió vacío: todo lo registrado estaba en memoria.

El lunes, su hijo quiso cargar la lista de 500 productos que le envió el distribuidor y la ventana se quedó blanca, con el mensaje «No responde», durante casi un minuto.

Antes de continuar, reflexione:

- ¿Dónde deberían guardarse los datos para que sobrevivan a un apagón?
- Si dos personas registran ventas a la vez, ¿qué problemas pueden aparecer?
- ¿Por qué cree que la ventana «no responde» mientras se importa la lista?
- ¿Qué haría un buen programa mientras realiza una tarea larga? Piense en las aplicaciones que usa en el celular.

Responda las preguntas anteriores en su cuaderno y compártalas con su pareja.

Actividad SQA: en su cuaderno o en el móvil, elabore una tabla de tres columnas —Qué sé, Qué quiero saber, Qué aprendí— sobre bases de datos y programación concurrente. Complete únicamente las dos primeras columnas por ahora; la tercera la completará al cierre de este material.

Lo que esperamos aprender: al terminar este material podrá guardar y consultar datos en una base relacional desde C#, y ejecutar tareas largas en segundo plano de forma segura y cancelable, y lo aplicará al caso de doña Martha.

B. Desarrollo central

Fundamentación científica

Unidad II · Acceso a bases de datos

Unidad II · 1. Introducción a la programación con bases de datos

Una base de datos relacional organiza la información en tablas formadas por filas (registros) y columnas (campos). Cada tabla tiene una clave primaria (PK) que identifica cada fila, y las relaciones entre tablas se expresan con claves foráneas (FK). El sistema gestor (SGBD) garantiza la integridad de los datos mediante restricciones.

En este curso usamos SQLite: un motor relacional que guarda toda la base en un único archivo (agrostock.db) y no requiere instalar un servidor, lo que lo hace ideal para aplicaciones de escritorio. Los conceptos y el código son equivalentes para MySQL o SQL Server: solo cambia el proveedor (MySqlConnection, SqlConnection).

Tabla	Campos principales	Relación
categorias	id (PK), nombre (UNIQUE)	Una categoría tiene muchos productos
productos	id (PK), codigo (UNIQUE), nombre, categoria_id (FK), precio (CHECK > 0), existencia (CHECK ≥ 0), vencimiento	Pertenece a una categoría
movimientos	id (PK), producto_id (FK), tipo (ENTRADA/SALIDA), cantidad, fecha	Muchos movimientos por producto

La aplicación no habla directamente con el archivo: usa un proveedor ADO.NET (Microsoft.Data.Sqlite) que traduce las llamadas de C# en instrucciones para el motor.

```
Formulario (UI)  →  ProductoRepository  →  Microsoft.Data.Sqlite  →  agrostock.db
   eventos              clases de acceso        proveedor ADO.NET         archivo SQLite
```

Ejemplo aplicado — AgroStock: el apagón de doña Martha ya no borra el inventario, porque los productos viven en el archivo agrostock.db y no en la memoria de la aplicación.

Unidad II · 2. El lenguaje SQL

SQL (Structured Query Language) es el lenguaje estándar de las bases relacionales. Se divide en DDL (definición: CREATE, ALTER, DROP) y DML (manipulación: INSERT, SELECT, UPDATE, DELETE). Las restricciones se declaran al crear la tabla.

```
CREATE TABLE productos (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,   -- clave primaria: identifica cada fila
    codigo       TEXT NOT NULL UNIQUE,                -- obligatorio y sin repetirse
    nombre       TEXT NOT NULL,
    categoria_id INTEGER NOT NULL REFERENCES categorias(id),  -- clave foránea: debe existir
esa categoría
    precio       REAL NOT NULL CHECK (precio > 0),    -- CHECK: regla que cada fila debe
cumplir
    existencia   INTEGER NOT NULL DEFAULT 0 CHECK (existencia >= 0),
    vencimiento  TEXT NULL                            -- yyyy-MM-dd; NULL si el producto no
vence
);
```

Operación CRUD	Sentencia SQL	Ejemplo
Create	INSERT	INSERT INTO productos (codigo, nombre, categoria_id, precio) VALUES ('FER-004','Sulfato',1,980);
Read	SELECT	SELECT codigo, nombre FROM productos WHERE existencia < 10 ORDER BY nombre;
Update	UPDATE	UPDATE productos SET precio = precio * 1.05 WHERE categoria_id = 1;
Delete	DELETE	DELETE FROM productos WHERE codigo = 'HER-005';

```
-- JOIN: nombre de la categoría junto al producto (une las tablas por la clave foránea)
SELECT p.codigo, p.nombre, c.nombre AS categoria
FROM productos p JOIN categorias c ON c.id = p.categoria_id;

-- GROUP BY: valor del inventario por categoría (agrupa y suma)
SELECT c.nombre, COUNT(*) AS productos, SUM(p.precio * p.existencia) AS valor
FROM productos p JOIN categorias c ON c.id = p.categoria_id
GROUP BY c.nombre ORDER BY valor DESC;
```

Cuidado: un UPDATE o DELETE sin WHERE afecta a todas las filas de la tabla. Escriba primero el SELECT con el mismo WHERE para ver qué filas cambiarán.

Ejemplo aplicado — AgroStock: «¿qué productos tienen existencia menor que 10?» y «¿cuánto vale mi inventario?», las preguntas que doña Martha respondía sumando a mano, ahora son una consulta SELECT.

Unidad II · 3. Clases para la gestión de bases de datos

ADO.NET define un conjunto de clases comunes a todos los proveedores. Con Microsoft.Data.Sqlite se llaman así:

Clase	Responsabilidad
SqliteConnection	Abre y cierra la conexión con el archivo de la base (cadena de conexión).
SqliteCommand	Contiene la sentencia SQL y sus parámetros. ExecuteReader (SELECT), ExecuteNonQuery (INSERT/UPDATE/DELETE), ExecuteScalar (un valor).
SqliteDataReader	Recorre las filas del resultado, hacia adelante y de solo lectura.
SqliteTransaction	Agrupa varias operaciones: se confirman todas (Commit) o ninguna (Rollback).

```
public List<Producto> Listar(string filtro = "")
{
    using var cn = Db.Abrir();          // using: la conexión se cierra SIEMPRE, aunque
ocurra un error
    using var cmd = cn.CreateCommand();
    cmd.CommandText = SelectBase + " WHERE p.nombre LIKE $f OR p.codigo LIKE $f ORDER BY
p.nombre";
    cmd.Parameters.AddWithValue("$f", $"%{filtro}%");   // el valor viaja aparte: nunca se
concatena

    var lista = new List<Producto>();
    using var rd = cmd.ExecuteReader(); // recorre las filas una por una
    while (rd.Read())
        lista.Add(new Producto { Id = rd.GetInt32(0), Codigo = rd.GetString(1), Nombre =
rd.GetString(2) /* ... */ });
    return lista;
}
```

Inyección SQL. Si se concatena lo que escribe el usuario, alguien puede alterar la consulta. Los parámetros envían el valor por separado y el motor nunca lo interpreta como código SQL.

```
// MAL: si el usuario escribe  ' OR 1=1 --  la condición siempre es verdadera y se muestran
TODOS los productos
cmd.CommandText = "SELECT * FROM productos WHERE nombre = '" + txtBuscar.Text + "'";

// BIEN: $f es un parámetro; el texto escrito se compara tal cual, aunque tenga comillas o
guiones
cmd.CommandText = "SELECT * FROM productos WHERE nombre = $f";
cmd.Parameters.AddWithValue("$f", txtBuscar.Text);
```

Patrón Repositorio. Todo el SQL vive en una clase (ProductoRepository) y los formularios solo llaman a sus métodos (Listar, Insertar, Actualizar, Eliminar). Así la interfaz no depende del motor de base de datos.

Ejemplo aplicado — AgroStock: si un cliente busca «O'Brien», o alguien escribe comillas en el cuadro de búsqueda, AgroStock no se rompe ni expone datos, porque la búsqueda usa un parámetro.

Unidad II · 4. Controles de acceso a bases de datos y CRUD

Windows Forms enlaza datos a controles mediante la propiedad DataSource. BindingSource actúa como intermediario: conoce el elemento actual (Current), permite filtrar y notifica cambios.

```
fuente.DataSource = repo.Listar(txtBuscar.Text);   // BindingSource: intermediario
dgv.DataSource = fuente;                             // DataGridView muestra la lista

cboCategoria.DataSource = repo.Categorias();
cboCategoria.DisplayMember = "Nombre";   // lo que el usuario VE
cboCategoria.ValueMember = "Id";         // lo que el programa GUARDA (la clave foránea, en
SelectedValue)

private void Nuevo()
{
    var p = new Producto();
    using var dlg = new FrmProducto(p, repo.Categorias());
    if (dlg.ShowDialog(this) != DialogResult.OK) return;
    try { repo.Insertar(p); Cargar(); }
    // 2067 = SQLITE_CONSTRAINT_UNIQUE: la base rechazó el código repetido; se avisa sin
cerrar el programa
    catch (SqliteException ex) when (ex.SqliteExtendedErrorCode == 2067)
    {
        MessageBox.Show(this, $"El código {p.Codigo} ya existe.", "No se guardó");
    }
}
```

Transacciones. Una entrada o salida de mercadería cambia dos tablas: agrega el movimiento y modifica la existencia. Si una falla, la otra no debe quedar guardada.

```
using var tx = cn.BeginTransaction();   // desde aquí nada es definitivo
// ... INSERT en movimientos y UPDATE de productos.existencia, ambos con cmd.Transaction = tx
...
tx.Commit();   // todo salió bien: se confirman las dos operaciones
// Si ocurre una excepción antes del Commit (por ejemplo CHECK existencia >= 0), al liberar
"tx" se hace Rollback solo.
```

Ejemplo aplicado — AgroStock: si doña Martha registra una SALIDA de 50 sacos y solo hay 24, la base rechaza el movimiento, se revierte la transacción y la existencia nunca queda negativa.

Unidad III · Programación concurrente

Unidad III · 5. Introducción a la programación concurrente

Un proceso es un programa en ejecución con su propia memoria. Un hilo (thread) es una secuencia de instrucciones dentro de un proceso; un proceso puede tener varios hilos que comparten la misma memoria. La programación concurrente consiste en organizar varias tareas que progresan al mismo tiempo, ya sea en paralelo (varios núcleos) o intercalándose.

Las aplicaciones Windows Forms tienen un único hilo de interfaz (UI thread) que procesa los eventos y dibuja la ventana —el [STAThread] que vio en el Encuentro 1—. Si un manejador de eventos tarda 30 segundos, durante ese tiempo la ventana no puede redibujarse ni responder: Windows la marca como «No responde». La solución es mover el trabajo pesado a otro hilo y devolver solo los resultados a la interfaz.

Ejemplo aplicado — AgroStock: cargar 500 productos en el hilo de la interfaz deja la ventana blanca durante casi un minuto; en un hilo de trabajo, doña Martha puede seguir moviendo la ventana y viendo el avance.

Unidad III · 6. Clases para trabajar con hilos

Clase / técnica	Cuándo usarla
Thread	Control directo de un hilo (nombre, prioridad, IsBackground). Útil para aprender los estados; costoso crear muchos.
ThreadPool	Conjunto de hilos reutilizables administrado por .NET.
Task + Task.Run	Opción recomendada: ejecuta trabajo en el ThreadPool y se puede esperar con await.
async / await	Espera una tarea sin bloquear el hilo de la interfaz; el código continúa en la UI al terminar.
BackgroundWorker	Componente histórico de Windows Forms (eventos DoWork, ProgressChanged, RunWorkerCompleted).
IProgress<T> / Progress<T>	Reporta avance desde el hilo de trabajo; el manejador se ejecuta en el hilo de la UI.

```
private async void BtnIniciar_Click(object? sender, EventArgs e)   // async void: solo en
manejadores de eventos
{
    cts = new CancellationTokenSource();
    var token = cts.Token;
    // Progress<int> se crea en el hilo de la UI: su manejador también corre allí y puede
```

```
tocar controles
    var progreso = new Progress<int>(pct => barra.Value = pct);
    try
    {
        await Task.Run(() => Importar(ruta, progreso, token));   // el trabajo corre en el
ThreadPool
        lblEstado.Text = "Importación terminada";              // después del await se
vuelve al hilo de la UI
    }
    catch (OperationCanceledException) { lblEstado.Text = "Cancelada"; }
}
```

Ejemplo aplicado — AgroStock: FrmImportar usa Task.Run para leer el CSV, IProgress para llenar la barra y await para volver a la ventana sin congelarla.

Unidad III · 7. Estados de un hilo

Estado (ThreadState)	Significado	Se llega con
Unstarted	Creado pero no iniciado	new Thread(...)
Running	Ejecutándose o listo para ejecutarse	Start()
WaitSleepJoin	Bloqueado esperando	Sleep(), Join(), lock ocupado
Stopped	Terminó su método	Fin del método del hilo

```
var hilo = new Thread(() => Thread.Sleep(1500)) { Name = "Trabajador" };
Console.WriteLine(hilo.ThreadState);   // Unstarted
hilo.Start();
Thread.Sleep(200);                      // se da tiempo a que el hilo llegue a su Sleep
Console.WriteLine(hilo.ThreadState);   // WaitSleepJoin
hilo.Join();                            // el hilo principal espera a que termine
Console.WriteLine(hilo.ThreadState);   // Stopped
```

Ejemplo aplicado — AgroStock: mientras la importación espera entre producto y producto, su hilo está en WaitSleepJoin; al terminar pasa a Stopped y libera sus recursos.

Unidad III · 8. Mecanismos de sincronización

Cuando dos hilos modifican el mismo dato sin coordinación ocurre una condición de carrera: la operación contador++ en realidad es leer, sumar y escribir, y los hilos pueden intercalarse perdiendo incrementos.

```
object candado = new();
lock (candado) { contador++; }              // sección crítica: un hilo a la vez
Interlocked.Increment(ref contador2);       // operación atómica, sin lock

using var semaforo = new SemaphoreSlim(2);   // máximo 2 tareas simultáneas
```

```
await semaforo.WaitAsync();
try { /* descarga */ }
finally { semaforo.Release(); }              // SIEMPRE devolver el permiso, aunque falle
```

Regla de oro de Windows Forms: solo el hilo de la interfaz puede tocar los controles. Desde otro hilo se usa IProgress<T>, Control.Invoke o se devuelve el resultado y se asigna después del await. Violar esta regla produce la excepción «Operación no válida a través de subprocesos».

Ejemplo aplicado — AgroStock: el contador de productos insertados y la lista de líneas rechazadas se protegen con Interlocked y lock, para que el total mostrado a doña Martha siempre coincida con lo guardado.

Unidad III · 9. Detención controlada de hilos

Un hilo nunca debe detenerse a la fuerza (Thread.Abort está obsoleto y no funciona en .NET moderno): puede dejar archivos abiertos o datos a medias. La detención controlada es cooperativa: la interfaz solicita la cancelación con CancellationTokenSource.Cancel() y el hilo revisa el token periódicamente, libera recursos y termina.

```
using var tx = cn.BeginTransaction();
for (int i = 0; i < lineas.Length; i++)
{
    if (token.IsCancellationRequested)             // la tarea revisa el token en un punto
seguro
    {
        tx.Rollback();                             // nada queda a medias
        token.ThrowIfCancellationRequested();      // sale de la tarea con
OperationCanceledException
    }
    repo.Insertar(Parsear(lineas[i]), cn, tx);
    progreso?.Report((i + 1) * 100 / lineas.Length);
}
tx.Commit();                                       // solo si llegó al final se vuelve
definitivo
```

Ejemplo aplicado — AgroStock: si el hijo de doña Martha pulsa «Cancelar» a la mitad de la importación, la transacción se revierte y la base queda exactamente como estaba, sin 250 productos a medias.

10. Integración: AgroStock v2

El código completo de AgroStock v2 está en la carpeta de recursos del Encuentro 2 (guía interactiva). Estructura del proyecto:

```
AgroStock.E02/
├── AgroStock.E02.csproj
├── Program.cs                ← llama a Db.Inicializar() antes de abrir la ventana
├── Models/Producto.cs        ← Producto y Categoria
├── Data/Db.cs                ← conexión y creación de la base
├── Data/ProductoRepository.cs ← CRUD parametrizado y movimientos
└── Forms/
    ├── FrmPrincipal.cs       ← DataGridView + BindingSource
    ├── FrmProducto.cs        ← ComboBox enlazado a categorías
    ├── FrmMovimiento.cs      ← entradas y salidas en una transacción
    └── FrmImportar.cs        ← Task.Run, IProgress, CancellationToken
agrostock.db · agrostock.sql · productos_importar.csv · DemoHilos/ (proyecto de consola)
```

La interfaz del Encuentro 1 casi no cambia: la lista en memoria (BindingList) se reemplaza por llamadas al repositorio.

Ejemplo aplicado — AgroStock: el cuaderno de doña Martha vs. AgroStock v1 vs. AgroStock v2.

Aspecto	Lista en memoria (v1, E1)	Base de datos SQLite (v2, E2)
Almacenamiento	BindingList: se pierde al cerrar o apagarse	Archivo agrostock.db: sobrevive al cierre y al apagón
Búsqueda	Filtro en memoria sobre la lista	Consulta SQL con LIKE y parámetros
Datos inválidos	Solo validación en el formulario	Validación + restricciones (UNIQUE, CHECK, FK) de la base
Existencias	Se edita el número a mano	Movimientos de ENTRADA y SALIDA en una transacción
Carga de 500 productos	Inviable: se escribe uno por uno	Importación en segundo plano, con progreso y cancelación

Ejercitación

Las siguientes actividades fomentan el aprendizaje autónomo e investigativo y se realizan de forma individual y colaborativa.

Actividad individual

- Complete la tercera columna de su tabla SQA («Qué aprendí») con las ideas clave de la fundamentación científica.
- Resuelva la autoevaluación sin mirar el material y luego verifique con la clave.

1. ¿Qué sentencia SQL corresponde a la operación Read del CRUD?

a) INSERT

b) SELECT

c) UPDATE

d) DELETE

2. ¿Por qué se usan parámetros ($f) en lugar de concatenar texto en SQL?

a) Porque son más cortos

b) Para evitar la inyección SQL y errores de formato

c) Porque SQLite no acepta comillas

d) No hay diferencia

3. ¿Qué método de SqliteCommand se usa para un INSERT sin resultados?

a) ExecuteReader

b) ExecuteNonQuery

c) ExecuteScalar

d) Read

4. ¿Por qué una ventana Windows Forms «no responde» durante una tarea larga?

a) Porque falta memoria

b) Porque el hilo de la interfaz está ocupado y no procesa eventos

c) Porque la base está cerrada

d) Por un virus

5. Un hilo que ejecuta Thread.Sleep(1000) está en el estado:

a) Unstarted

b) Running

c) WaitSleepJoin

d) Stopped

6. Dos hilos hacen contador++ un millón de veces cada uno sin lock. El resultado final:

a) Siempre es 2 000 000

b) Puede ser menor que 2 000 000

c) Siempre es 1 000 000

d) Lanza una excepción

7. ¿Cuál es la forma correcta de detener una tarea en .NET?

a) Thread.Abort()

b) Cerrar la aplicación

c) Solicitar la cancelación con CancellationToken y que la tarea la revise

d) Poner el hilo en null

Actividad colaborativa

El trabajo interencuentro se retoma en la BOA 2. Debe realizarse en grups (salvo las tareas individuales) y entregarse en el Encuentro 3.

1. Corregir AgroStock v1 con la retroalimentación del corte 1 (tarea 1).
2. AgroStock v2: CRUD sobre SQLite y tabla de movimientos (tarea 3).
3. Importación en segundo plano con progreso, cancelación y rollback, y un video de máximo 2 minutos (tarea 4).
4. Individual: resolver las 10 consultas SQL (tarea 2), redactar el informe Thread / Task / BackgroundWorker (tarea 5) y leer la sección «¿Qué es un ORM?» del folleto del Encuentro 3 (tarea 6).

Práctica de programación

Resuelva de forma individual los siguientes ejercicios. Ejecute agrostock.sql en DB Browser for SQLite y escriba cada consulta en su script .sql. Las 10 consultas son obligatorias (tarea 2 de la BOA 2).

Nivel 1 – Básico

1. Listar todos los productos ordenados por precio de mayor a menor.
2. Mostrar código, nombre y existencia de los productos con existencia menor que 10.
3. Mostrar los productos que vencen antes del 31/12/2026.
4. Mostrar cada producto con el nombre de su categoría (JOIN).
5. Contar cuántos productos hay por categoría.

6. Reto de programación 1: agregue a AgroStock v2 un ComboBox para filtrar por categoría.

Nivel 2 – Intermedio

1. Calcular el valor total del inventario (precio × existencia).
2. Insertar una nueva categoría «Riego» y un producto de esa categoría.
3. Aumentar 5 % el precio de los fertilizantes (verifique primero con SELECT).
4. Registrar una SALIDA de 3 unidades del producto HER-004 en movimientos.
5. Eliminar los productos con existencia 0 de la categoría Herramientas (verifique primero con SELECT).
6. Reto de programación 2: cree MovimientoRepository y un formulario para registrar entradas y salidas que actualice la existencia en la misma transacción.

Nivel 3 – Avanzado (opcional)

1. Reto de programación 3: en DemoHilos, cree una demostración productor–consumidor con BlockingCollection<T>.

Entrega

Suba al aula virtual el script .sql con las 10 consultas y sus resultados (tarea «Ejercitación E2»), el proyecto AgroStock v2 —con la importación en segundo plano y el video— sin las carpetas bin y obj, y el informe de una página en PDF. Si usó ejemplos o IA, indíquelo en el README.

Lista de verificación: marque lo que ya logró.

☐ Instalé DB Browser for SQLite y ejecuté agrostock.sql

☐ Resolví las 10 consultas SQL

☐ Implementé ProductoRepository con consultas parametrizadas

☐ El CRUD funciona desde la interfaz y registré movimientos

☐ Ejecuté DemoHilos y observé la condición de carrera

☐ La importación no congela la ventana

☐ Puedo cancelar la importación y se revierte

☐ Entregué AgroStock v2, el video y el informe

C. Cierre y consolidación del aprendizaje

Complementación

El dominio de las bases de datos y de la programación concurrente va más allá de memorizar sentencias o nombres de clases. Reflexione sobre las siguientes preguntas, que conectan lo aprendido con su realidad como futuro(a) ingeniero(a) en sistemas:

Preguntas para reflexionar

- **Conexión con la práctica:** piense en los negocios de su comunidad —un agroservicio, una pulpería, una cooperativa—. ¿De qué manera una base de datos y una importación en segundo plano pueden reemplazar el cuaderno y reducir los errores de registro?
- **Transferencia del aprendizaje:** ya sabe guardar datos con SQL parametrizado y proteger la interfaz con tareas en segundo plano. ¿Cómo podría aplicarlo a planillas, facturación o control de ventas?
- **Relación con el eje Madre Tierra:** ¿cómo puede digitalizar el inventario reducir el uso de papel, las pérdidas de insumos vencidos y los viajes innecesarios de los productores? Escriba tres ideas.

Sitios web para ampliar sus conocimientos

Le comparto una selección de recursos en línea, de fuentes confiables, para que pueda seguir perfeccionando sus habilidades:

Recurso	Descripción	Enlace
Microsoft Learn — Microsoft.Data.Sqlite: información general	Cómo conectarse a SQLite desde .NET: conexiones, comandos, parámetros y transacciones.	https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/
SQLite — SQL As Understood By SQLite	Referencia oficial del lenguaje SQL que entiende SQLite.	https://www.sqlite.org/lang.html
Microsoft Learn — Programación asincrónica con async y await	Cómo esperar tareas largas sin bloquear la interfaz.	https://learn.microsoft.com/es-es/dotnet/csharp/asynchronous-programming/
Microsoft Learn — Cancelación en subprocesos administrados	Cancelación cooperativa con CancellationToken.	https://learn.microsoft.com/es-es/dotnet/standard/threading/cancellation-in-managed-threads

Glosario del encuentro

Término	Definición
Clave primaria (PK)	Campo que identifica de forma única cada fila de una tabla.
Clave foránea (FK)	Campo que referencia la clave primaria de otra tabla y crea la relación.
Restricción	Regla de la base de datos (NOT NULL, UNIQUE, CHECK) que protege la integridad de los datos.
CRUD	Create, Read, Update, Delete: las cuatro operaciones básicas sobre datos.
Parámetro	Marcador (como $f) cuyo valor viaja separado del SQL; evita la inyección.
Inyección SQL	Ataque que altera una consulta concatenando texto malicioso; se evita con parámetros.
Repositorio	Clase que concentra todo el SQL de una entidad y oculta el motor a la interfaz.
Transacción	Grupo de operaciones que se confirman todas (Commit) o ninguna (Rollback).
Hilo	Secuencia de ejecución dentro de un proceso; varios hilos comparten memoria.
Hilo de la interfaz	Único hilo que dibuja la ventana y puede tocar los controles.
Condición de carrera	Error que aparece cuando varios hilos modifican un dato sin sincronización.
lock	Instrucción de C# que permite a un solo hilo a la vez ejecutar una sección crítica.
Interlocked	Operaciones atómicas (como Increment) que no necesitan lock.
CancellationToken	Señal para solicitar la detención cooperativa de una tarea.

Cierre y transferencia del aprendizaje

Guardar datos y hacer trabajo en segundo plano no son solo técnicas: son decisiones sobre la confianza que el usuario puede tener en su programa. Una base de datos con restricciones, consultas parametrizadas y tareas cancelables pueden convertir el inventario de doña Martha en una herramienta confiable, que resiste apagones, errores de captura y listas de cientos de productos.

Así como doña Martha pasó del cuaderno a la pantalla, y ahora de la memoria a una base de datos, usted avanza en su camino en la programación de aplicaciones de escritorio: AgroStock v2 es la segunda etapa de una aplicación que completaremos en el próximo encuentro.

Conclusiones clave del encuentro

1. Los datos viven en la base, no en la memoria: sobreviven al cierre y a los apagones.
2. Toda consulta con datos del usuario se parametriza; el SQL vive en el repositorio.
3. El hilo de la interfaz solo dibuja y responde; el trabajo pesado va a otro hilo o tarea, con progreso por IProgress.
4. La sincronización (lock, Interlocked, SemaphoreSlim) protege los datos compartidos entre hilos.
5. La cancelación cooperativa con CancellationToken y una transacción con Rollback detienen una tarea sin dejar datos a medias: así AgroStock v2 reemplaza la lista en memoria de v1.

Transferencia del aprendizaje

Los conocimientos que ha adquirido son un pilar que trasciende este componente curricular. Los principios de acceso a datos y concurrencia son directamente transferibles a su práctica profesional:

1. SQL y las restricciones de integridad son los mismos en MySQL, SQL Server o PostgreSQL: solo cambia el proveedor.
2. El patrón Repositorio y los parámetros le servirán para el ORM que verá en el Encuentro 3.
3. Cualquier aplicación que consuma un servicio web o genere un reporte pesado necesitará las mismas tareas en segundo plano y cancelación.

“La práctica constante es la clave”. Aplíquese a las consultas del encuentro y a AgroStock v2, y estará construyendo una competencia que le acompañará durante toda su carrera profesional.

Referencias

Ceballos Sierra, F. J. *Enciclopedia de Microsoft Visual C#*. Editorial RA-MA.

Deitel, H. M. y Deitel, P. J. *C# Cómo programar*. Editorial Pearson, Prentice Hall.

Microsoft (2024). *Guía de escritorio (Windows Forms .NET)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

Microsoft (2024). *Información general sobre eventos (Windows Forms .NET)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/forms/events

Microsoft (2024). *Microsoft.Data.Sqlite: información general*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/

Microsoft (2024). *Subprocesamiento administrado y programación asincrónica*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/

Microsoft (2024). *Tutorial: Creación de una aplicación de Windows Forms con .NET*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/get-started/create-app-visual-studio

SQLite Consortium (2024). *SQL As Understood By SQLite*. https://www.sqlite.org/lang.html



¡Universidad
del Pueblo y para el Pueblo!

www. unan.edu.ni